

Symmetry Breaking versus Zero Initialization

[week-module-3](#), [coursera-platform](#)

[paulinpaloalto](#) 1 June 21, 2021, 10:47am

When we implemented Logistic Regression in Week 2 of Course 1, Prof Ng tells us that it suffices to use zero initialization for the w and b values that will be learned through Gradient Descent and we will still get a valid solution. But when we get to real Neural Networks in Week 3 with the 2 layer net, we can no longer get away with using zero initializations and need to use random initializations of all the $W^{[1]}$ values in order to “break symmetry”. But Prof Ng only mentions this briefly and doesn’t really get into much detail. Frequently people ask why Logistic Regression is different in this respect. As always, it comes back to the math.

Let’s start by looking at the formula for the gradient of w in Logistic Regression:

$$dw = \frac{1}{m} X \cdot (A - Y)^T$$

If we start with w and b all zeros, then A will be 0.5 for all samples, since $\text{sigmoid}(0) = 0.5$. The Y values are all either 0 or 1 (the labels), so you end up with a non-zero values for dw . That means that Gradient Descent can learn a different value for w even starting with w and b as zeros. It works similarly for b .

Now look at the formulas for the shallow two layer NN case in DLS C1 W3 and things are quite a bit more complicated:

$$dZ^{[2]} = A^{[2]} - Y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} \cdot A^{[1]T}$$

$$dZ^{[1]} = \left(W^{[2]T} \cdot dZ^{[2]} \right) * g'^{[1]}(Z^{[1]})$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} \cdot X^T$$

Starting with both W and both b values zero, you can see that $A^{[1]}$ will be all zeros because $g^{[1]}$ is $\tanh$ and $\tanh(0) = 0$. So that will result in $dW^{[2]}$ being zero. Then $W^{[2]}$ is all zeros, so $dZ^{[1]}$ will be zero and that means $dW^{[1]}$ will also end up being zero. So Gradient Descent is stuck: it can’t change either of the W matrices, meaning that no learning can take place.

Note that there are some further subtleties to explore here: e.g. what if we changed the layer one activation to something like *sigmoid* that doesn’t give 0 output for 0 input, would that change the result? It turns out it does not. See below for more on that.

It actually turns out that you have a choice of the method to “break symmetry” here: you can do as Prof Ng recommends and randomly initialize all the W matrices, but set the biases to zero initially. Or you can do the reverse: set the W values all to zero, but randomly initialize the bias values. In either case symmetry will be broken and Gradient Descent can learn.

What about using non-zero initializations but making them all the same?

The next question is what happens if we use an activation function in the hidden layer for which $g(0) \neq 0$? Or we could initialize the weights and biases with the same small values as in:

```
W1 = np.ones((n1, n0)) * 0.01
```

It turns out that what happens if you use that style of initialization is that you *can* learn new values of all the $W^{[l]}$ and $b^{[l]}$ parameters, but the values that you get are themselves symmetric in the sense that all the neurons output the same values at every level. That means that it's equivalent to a network with just a single neuron. So it's not just that there's something bad about starting with zero values: what is bad is starting with all the *same* values, either zero or non-zero. It really is the case that you need to “break symmetry” by starting with values that are all different at least for the $W^{[l]}$ values. We could dig deeper and prove this mathematically, but it's easy enough and kind of fun just to try it yourself and watch what happens. I used the Week 4 Application assignment code and then added my own “init” routine using the technique above and then ran 200 iterations and printed out all the $W^{[l]}$ and $b^{[l]}$ values and it's easy to see what happens!

[Weight matrix initialization](#)

[Initialization of Weights in Logistic and Linear Regression](#)

[W2 A2 Ex-8 Value Error for w](#)

[Parameter Initializatio](#)

[Questions about initialization](#)

[Training of transformer](#)

[Week 3 activation function](#)

[Week 3 Random Initialization](#)

[Course 2 Week 1 PA 1: Why does zero weight cause no change in loss](#)

[Glorot Initializer for Bias?](#)

[FAQ: Frequently Asked Questions for all DLS Courses](#)

[Zeros initialization for weights matrices](#)

[Week 1 - Programming Assignment 1](#)

[Week 1, Programming Assignment initialization, Exercise 1 - initialize parameters zeros](#)

[C2W1 Individual Neurons and Classification](#)

[Initializing weights with 0](#)

[Neural Networks Hidden Layers](#)

[Randomly initialize parameter b instead of W](#)

[Why do we multiply the random initial weights by 0.01?](#)

[Initial Parameter Values in Neural Networks \(Deep Learning Special Course, Course 1 Week 3\)](#)

[Initializing Weights](#)

[W3 Initialization with identity matrix](#)

[Neural Networks Weights](#)

[Why zero initializations fails?](#)

[Orchestration of units in a layer: Who tells each unit what to look for?](#)

[Why don't weights get adjusted when initialized to 0?](#)

[What is the effect random initialization of W on multiple nodes and in neural network when all of them are doing the same thing](#)

[Clarification on Zero Initialization in Neural Network Linear Regression](#)

[How \$\tanh\(0\)\$ is zero](#)

[Can the random initialization of weights return very small values using `np.random.randn\(\(x,y\)\)\*0.001`?](#)

[Weight Initialization](#)

[How is training finding different weights for ReLU units that are all initialized to 0?](#)

[Ask a question about neural networks](#)

[How to decide the initial value of weight and bias?](#)

[What makes different neurons calculate different parameters within a layer?](#)

[Pytorch initialisation](#)

[Week 1, Optional Lab 02: Question about TensorFlow](#)

[Random Initialization in Neural Networks](#)

[Logistic Regression fundamental question](#)

[First lesson, 4 questions about NN](#)

[Week 1, Programming Assignment initialization, Exercise 1 - initialize parameters zeros](#)

[Zero initialization of weights](#)[Week 3 Programming Assignment Exercise 3 Error](#)[Doubt in concept of neural network representation \(week3 video3\)](#)[Concept in Initialization Assignment-Help needed in understanding](#)[Activation Functions, Weights, and Biases of Each Layer](#)[Extra weights confusion](#)[Hidden layer first iteration neural network](#)[1157350959](#) 3 June 28, 2021, 5:21pm

Hi! Your explanation helps a lot! Could you explain more on why using a sigmoid which “doesn’t give 0 output for 0 input” still can’t change the result? Is it still about math?

[paulinpaloalto](#) 4 June 28, 2021, 5:26pm

Yes, all this is about math. Once the output values from the hidden layers are not zero anymore, the math gets a bit more complicated. What is the case is the rows of the weights are all equal within any given layer and the result is that the outputs are all equal at each given level. Then the gradients also have all the rows equal, so that the row-based symmetry is preserved as you run back propagation. That gives the result that I described: all the neurons throughout the network at all layers all give the same output at the given layer, so it’s equivalent to having only one neuron at every layer.

Rather than trying to prove that mathematically, I suggest you do what I said in my earlier post: just try it and watch what happens. Print the values of $W^{[l]}$ and $b^{[l]}$ after 100 and 200 iterations. It will be very clear that what I said above is true. You’ll see that they change meaning that learning is happening, but the rows stay equal. So even though the gradients are non-zero, the learning is not actually useful. You could consider this the analog of the famous truism: “A picture is worth a thousand words.” Seeing the results is better than 100 lines of mathematical proof.

If you do try this, it’s easier not to change the activation function. Just do the more general thing of using the “uniform non-zero” initialization style. So you only need to change the “init” routine to do something like I suggested above:

```
W1 = np.ones((n1, n0)) * 0.01
```

For all layers, of course. Then just run the training for 100 or 200 iterations (no point in waiting for a larger number) and print the W and b values at all layers.

Note that you can even make the symmetric values be different at each layer and you still get the same symmetric result. E.g. you could add the layer number as a factor in the above initialization:

$$W^{[l]} = \text{np.ones}((n^{[l]}, n^{[l-1]})) * 0.01 * l$$

If you try it that way, you'll see that you still have the same problem that the symmetry at each layer is preserved.

[Divy.Kala](#) 5 August 9, 2021, 6:58am

I tried running gradient descent with initial random biases, and 0 weights. I tried a few different learning rates, tried scaling the biases from 10^{-3} to 100, and also ran gradient descent for up to 150,000 iterations.

I found that the weights were changing so slowly that they virtually had no effect on predictions. The model accuracy remained 50%. Did anyone else observe something similar?

[paulinpaloalto](#) 6 August 9, 2021, 8:16pm

Hi, Divy.

It's great that you are doing that. You always learn something interesting when you try your own experiments! Which exercise did you use as the vehicle for this experiment? I just tried it using the Planar Data exercise in C1 Week 3. My results were just as good as with the original initialization when I used the initialization strategy you describe: set the W values to 0 and then use the normal distribution scaled by 0.01 for the bias values.

Also did you use all the other code in the notebook as written? Or are you doing this in your own environment?

[paulinpaloalto](#) 7 August 10, 2021, 3:54am

I also tried this experiment with the 2 layer model in Course 1 Week 4 Assignment 2. Just using the normal distribution scaled by 0.01 for the bias vectors and not changing any of the other hyperparameters, I get 97.6% training accuracy after 2500 iterations and 66% test accuracy. Not that much worse than with the usual initialization method. But I did notice some oscillation of the cost in the last few hundred iterations. We could probably fiddle with the learning rate and number of iterations and get a better result.

Related topics

Topic	Replies	Activity
Week 1, Programming Assignment initialization, Exercise 1 - initialize parameters zeros coursera-platform	8	December 15, 2023
Random Initalization in Neural Networks week-module-3, coursera-platform	15	September 11, 2024
Randomly initialize parameter b instead of W coursera-platform	6	August 23, 2022
Week 3 Random Initialization coursera-platform	6	May 6, 2022
Course 2 Week 1 PA 1: Why does zero weight cause no change in loss coursera-platform	2	January 5, 2022